

AI4012 — Assignment #1

Due Date: Friday, September 18th.

Assignments are to be done individually. No late assignments will be accepted. Submit the assignment/code on Google Classroom.

Autonomous intelligent agents operating in complex domains (e.g., algorithmic finance, dynamic pricing, risk monitoring) require continuous, high-integrity data streams. In this assignment, you will engineer a production-grade data sourcing, consensus, and serving pipeline. Your system must ingest live micro-data from online sources, perform cross-source consensus verification, track provenance/quality, enforce time-to-live (TTL) storage and cleanup in PostgreSQL (JSONB), and serve standardized REST payloads optimized for agent runtimes.

1 System Architecture & Functional Requirements

Each one of you should select a single task from the spreadsheet provided. Your implementation must satisfy five core technical requirements:

1.1 1. Live Micro-Data Ingestion & Multi-Source Consensus

- **Ingestion/Scraping:** Continuously fetch or scrape micro-data live from at least two distinct online providers for your assigned task.
- **Consensus Verification:** Implement cross-source consensus (e.g., weighted average, outlier rejection, or majority voting) to compute a single source of truth alongside a dynamic *confidence* and *quality score*.
- **No Silent Fallbacks:** System errors (e.g., rate limits, bad schema, upstream downtime) must be explicitly propagated with clean error structures. Silent fallbacks or stale data masquerading as fresh are strictly disallowed.

1.2 2. Standardized Agent Servicing Payload

Data served to consuming agents must be serialized into a uniform JSON response adhering to data lineage, freshness, trust, and auditability guarantees. See Section 3 for the canonical specification.

1.3 3. Storage Engine & Lifecycle Management (PostgreSQL + JSONB)

- **Schema Design:** Store raw incoming payloads, calculated consensus, and full audit metadata in PostgreSQL using indexed JSONB columns.
- **TTL & Automated Purging/Summarization:** Implement an automated process (via background worker or database triggers/CRON) that purges expired records according to defined TTLs or summarizes fine-grained micro-data into historical rollups.

1.4 4. SLA & Reliability Testing

Your submission must include automated test suites validating:

- **API Availability:** High availability under sustained polling.
- **Latency Profile:** p_{95} response latency < 200 ms.
- **Freshness SLA:** Enforcement of maximum allowable data age ($\text{age_seconds} \leq \text{ttl_seconds}$).
- **Source Failover:** Seamless degradation and recalculation when one provider fails.

2 Task Specification Structure

Students will select a specific domain task from the provided spreadsheet. The following illustrates the canonical mapping for the Financial FX Rate benchmark:

Category	Sub-Domain	Task Description	HTTP Method & Path
Finance & Banking	FX Spot Market	Retrieve current FX spot rate	GET /v1/finance/fx

- **Request Input (Query Parameters):** `{"base": "USD", "quote": "EUR"}`
- **Raw Ingest Sample:** `{"base": "USD", "quote": "EUR", "rate": 0.92, "as_of": "2026-08-17T10:00:00Z"}`
- **Update Interval / Freshness:** Sub-minute updates (≈ 5 – 10 seconds).

3 Standardized Response Specification

Your REST API must yield responses conforming to this exact schema structure:

```
{
  "data": {
    "base": "USD",
    "quote": "EUR",
    "rate": 0.92,
    "as_of": "2026-08-17T10:00:00Z",
    "source": "provider"
  },
  "meta": {
    "request_id": "req_01J8FX7K3M2Q9P4R6T8V",
    "product_id": "finance.fx.spot.v1",
    "version": "1.0.0",
    "served_at": "2026-08-17T10:00:02Z",
```

```

"source_last_updated_at": "2026-08-17T09:55:00Z",
"freshness": {
  "age_seconds": 300,
  "ttl_seconds": 3600,
  "stale": false
},
"provenance": [
  {
    "source_id": "SRC-FX-001",
    "publisher": "Official FX Rate Provider",
    "retrieved_at": "2026-08-17T09:55:00Z"
  }
],
"trust": {
  "confidence": 0.96,
  "quality_score": 0.97,
  "verified": true
},
"license": {
  "type": "commercial",
  "usage": "agent_runtime"
},
"api": {
  "latency_ms": 142,
  "rate_limit": {
    "limit": 100,
    "window_seconds": 60
  }
},
"warnings": []
}

```

4 Submission

Your submission must include:

- The entire code to and scripts to run your implementation
- The test suite that you have run to test your system
- A two page report, one-page for the system architecture (in the simplest possible form) and one detailing the use of AI (models and prompts you used)

5 Grading Rubric

Criterion		Description	Weight
Specification	Compliance	Full alignment with JSON schema, metadata fields, and correct tracking of lineage, freshness, and trust metrics.	30%
Data Integrity & Consensus		Robust multi-source ingestion, cross-source consensus verification, and clean error handling (no silent fallbacks).	25%
Storage & Lifecycle		Efficient PostgreSQL JSONB schema, indexing, and automated TTL purging or data summarization mechanisms.	20%
SLA & Performance		Verified API availability, source failover handling, and p_{95} latency compliance under test loads.	15%
Code	Architecture & Testing	Conciseness and clarity of code, modularity, and automated integration/unit tests.	10%

Honor Policy

This assignment is a group learning opportunity that will be evaluated based on your ability to think as a group, work through a problem in a logical manner and implement a software program on your own. You may however discuss verbally or via email the general nature of the conceptual problem to be solved with your course instructor, but you are to complete the actual programming for this assignment without resorting to help from any other person or other resources that are not authorized as part of this course. If in doubt, ask the course instructor. You may not use the Internet to search for solutions to the problem.